Introduction to Approximation Algorithms

Overview

Approximation algorithms don't always return an optimal solution, but their solutions are often close to optimal. They always run in polynomial time, and they are often relatively simple.

Definition of α-approximation algorithm

A α-approximation algorithm is a polynomial-time algorithm that always returns a solution whose value ALG is within a factor α of the optimal value OPT.

8.1 Vertex Cover

Problem Statement

Given an undirected graph G, find a minimum vertex cover of G.

Algorithm

Set S = ∅. If G has an edge e = {u, v}, add both endpoints of e to S, and remove u and v from G. Repeat this process until G has no edges and return S.

Correctness

The edges considered by the algorithm form a matching M, so |ALG| = 2 · |M|. Furthermore, each vertex in OPT can cover at most one edge in M, so |OPT| ≥ |M|. Thus, |ALG| = 2 · |M| ≤ 2 · |OPT|.

8.2 Load Balancing

Problem Statement

Given n jobs and m machines, assign each job to a machine to minimize the maximum load.

Algorithm

The algorithm is greedy: iterate through the jobs (in any order), assigning each one to the machine with the smallest load so far.

Correctness

Suppose ALG achieves a makespan of T, the load of $i \in [m]$ is T, and k was the last job assigned to i. When ALG assigned k to i, i had the smallest load. This means the load on every machine is at least T – ℓ[k], so `$\sum_{j=1}^n \ell[j] \geq m(T - \ell[k])$`. Rearranging yields `$T \leq`

`\frac{\sum_{j=1}^n \ell[j]}{m} + \ell[k]$` .

Vertex Cover Approximation Algorithm

Algorithm Steps

1. Initialize an empty set S.
2. While the graph G has edges, do the following:
    - Select any edge e = {u, v} from G.
    - Add both u and v to S.
    - Remove u, v, and all edges incident to them from G.
3. Return the set S.

Analysis

The algorithm provides a 2-approximation for the vertex cover problem.

Proof

- The algorithm terminates because each iteration removes at least one edge.
- The set S is a vertex cover because every edge is considered and its endpoints are added to S.
- The size of S, |S|, is at most twice the size of the optimal vertex cover, OPT.

Load Balancing Approximation Algorithm

Algorithm Steps

1. Initialize the load of each machine to 0.
2. Iterate through each job in any order.
3. For each job, assign it to the machine with the smallest current load.

Analysis

The algorithm provides a 2-approximation for the load balancing problem.

Proof

- Let T be the maximum load achieved by the algorithm.
- Let ℓ[k] be the length of the last job assigned to the machine with maximum load T.
- At the time of assignment, the load of the machine was the smallest, so `$T - \ell[k] \leq \text{load of any other machine}$` .
- Summing over all machines gives `$\sum_{j=1}^n \ell[j] \geq m(T - \ell[k])$` .
- This leads to `$T \leq \frac{\sum_{j=1}^n \ell[j]}{m} + \ell[k]$` .
- Since `$\ell[k] \leq T^*$` , where `$T^*$` is the optimal makespan, `$T \leq 2T^*$` .